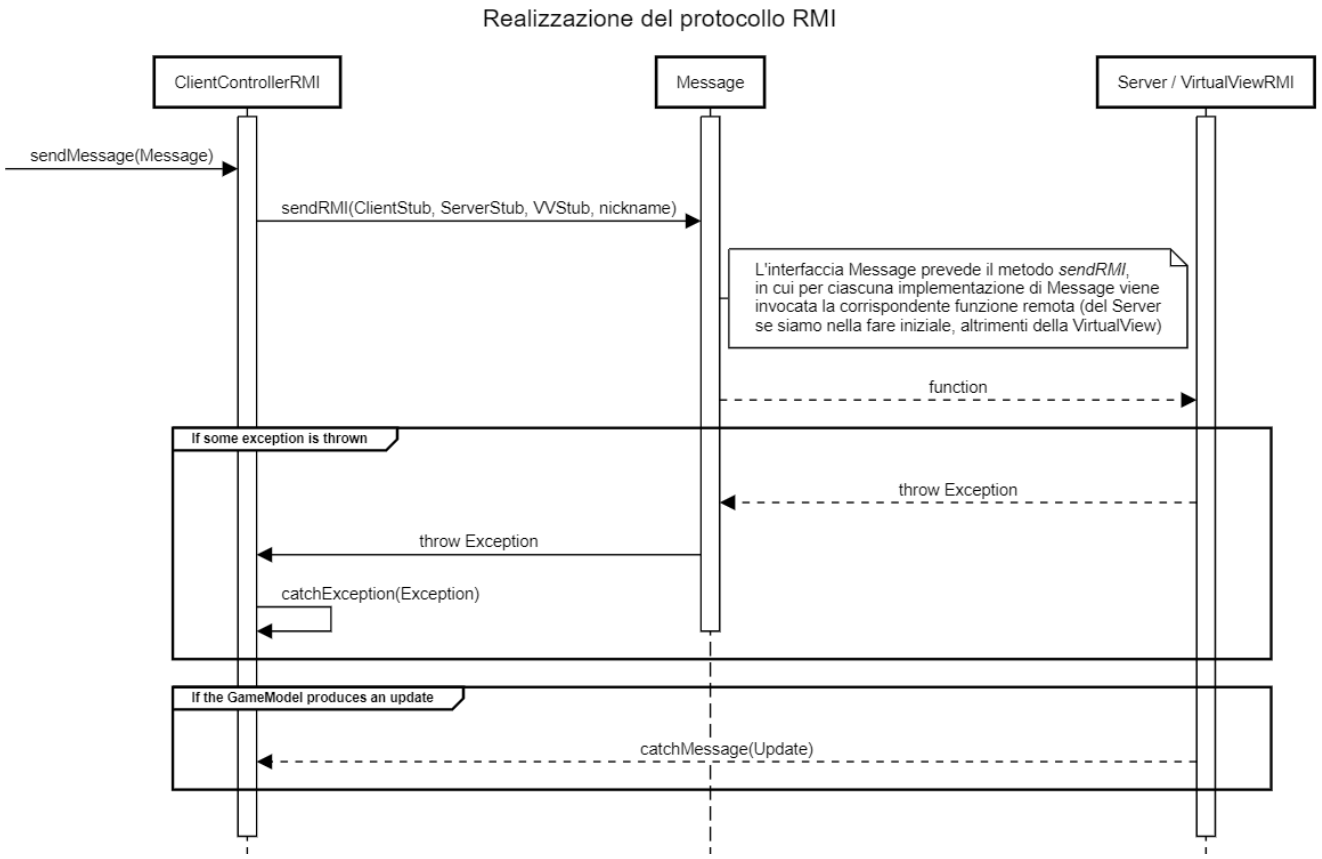
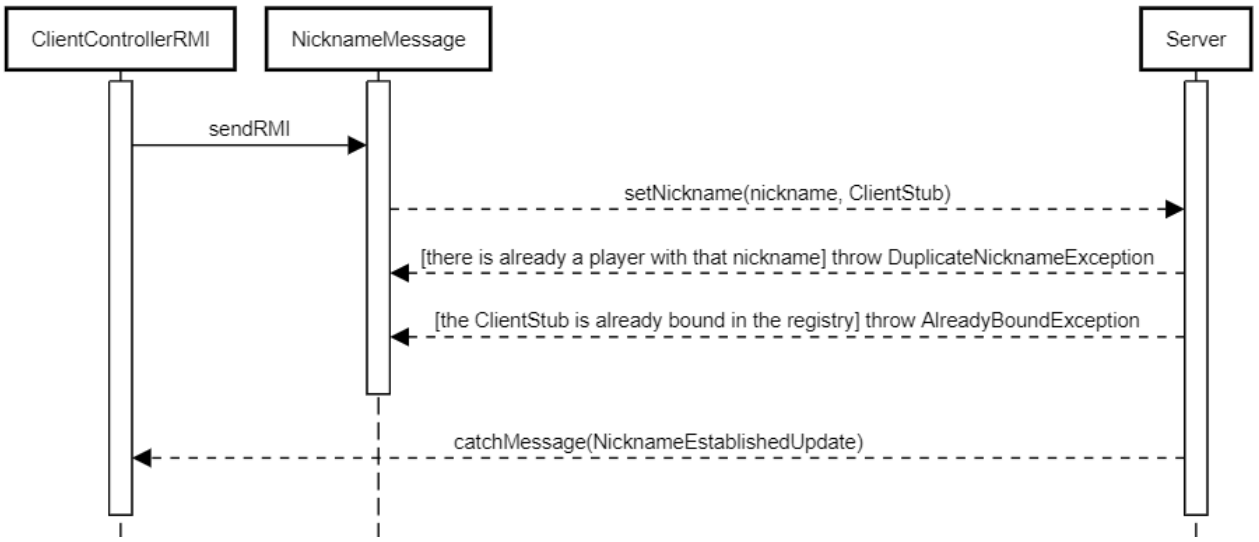
Protocollo RMI



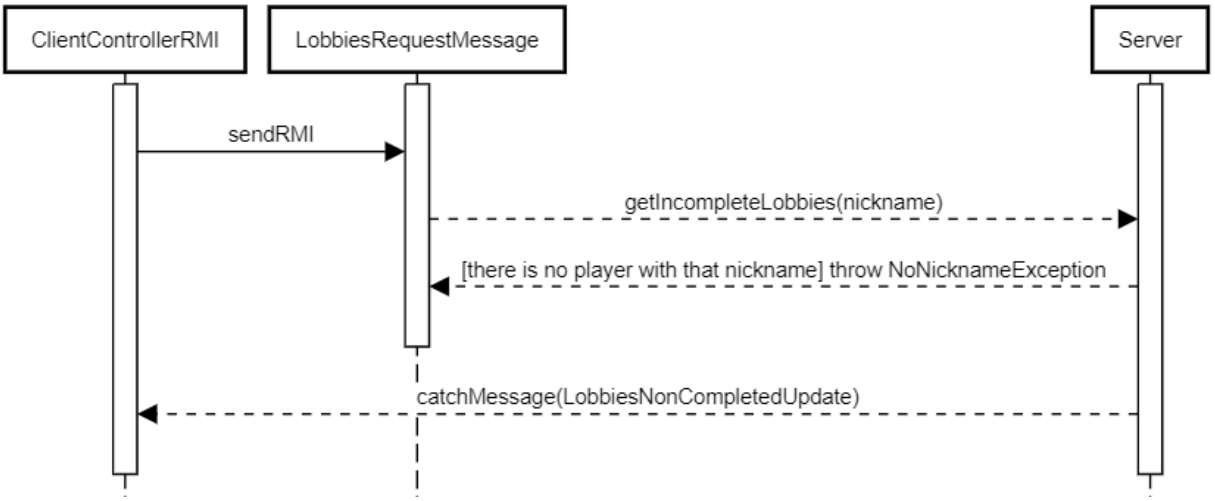
Sul RMI registry è definito il Server con il nome “CodexServer”. Quando il ClientController invoca per la prima volta una funzione remota del server (interfaccia ServerStub), passa un riferimento a sé stesso: in questo modo il server può caricare il client sul RMI registry e invocare le funzioni remote dell’interfaccia ClientStub.

I diagrammi seguenti mostrano nel dettaglio quale funzione remota viene invocata da ciascun tipo di messaggio, quali eccezioni vengono lanciate in caso di errore e quale Update viene mandato in caso di successo.

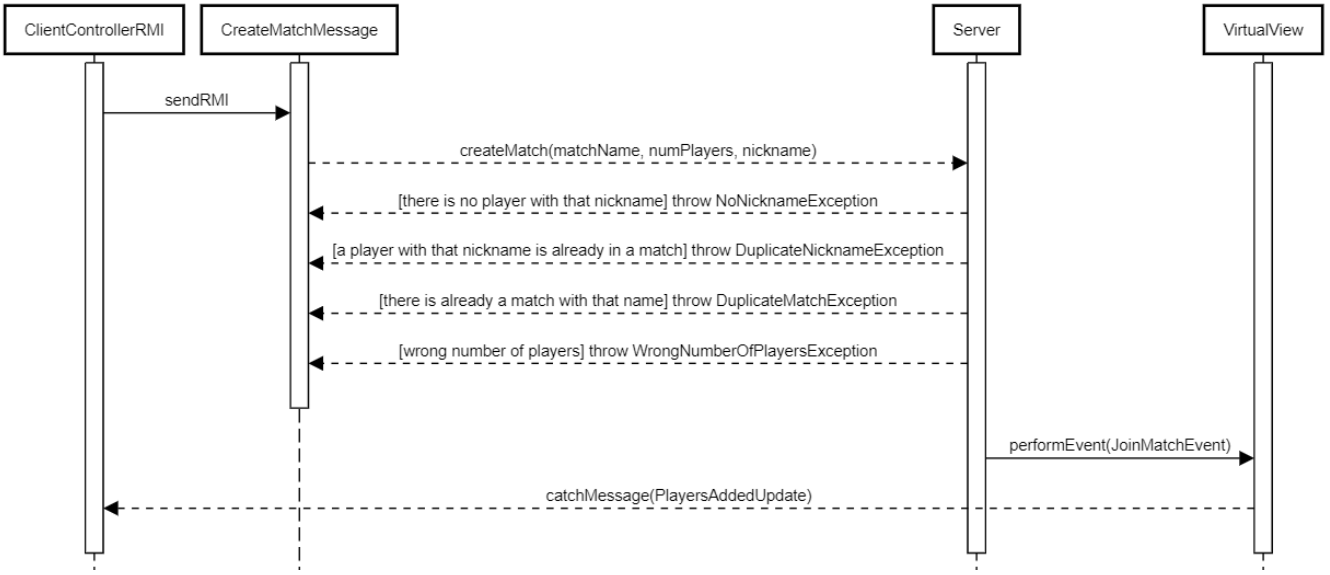
Set nickname



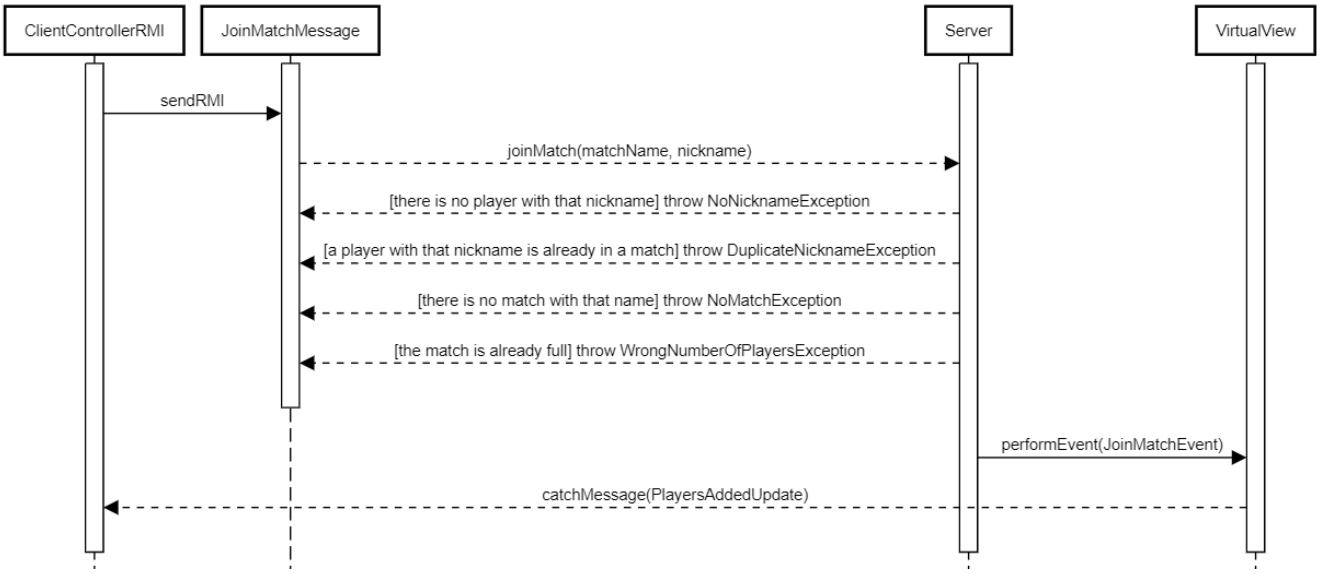
Get Incomplete Lobbies



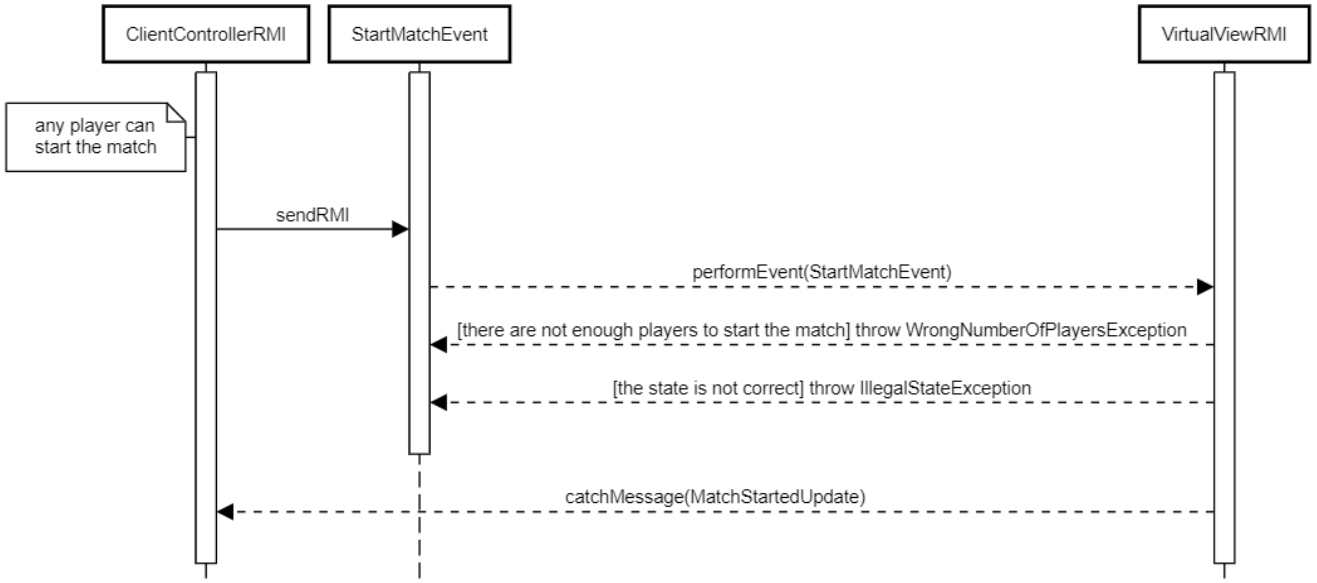
Create a match



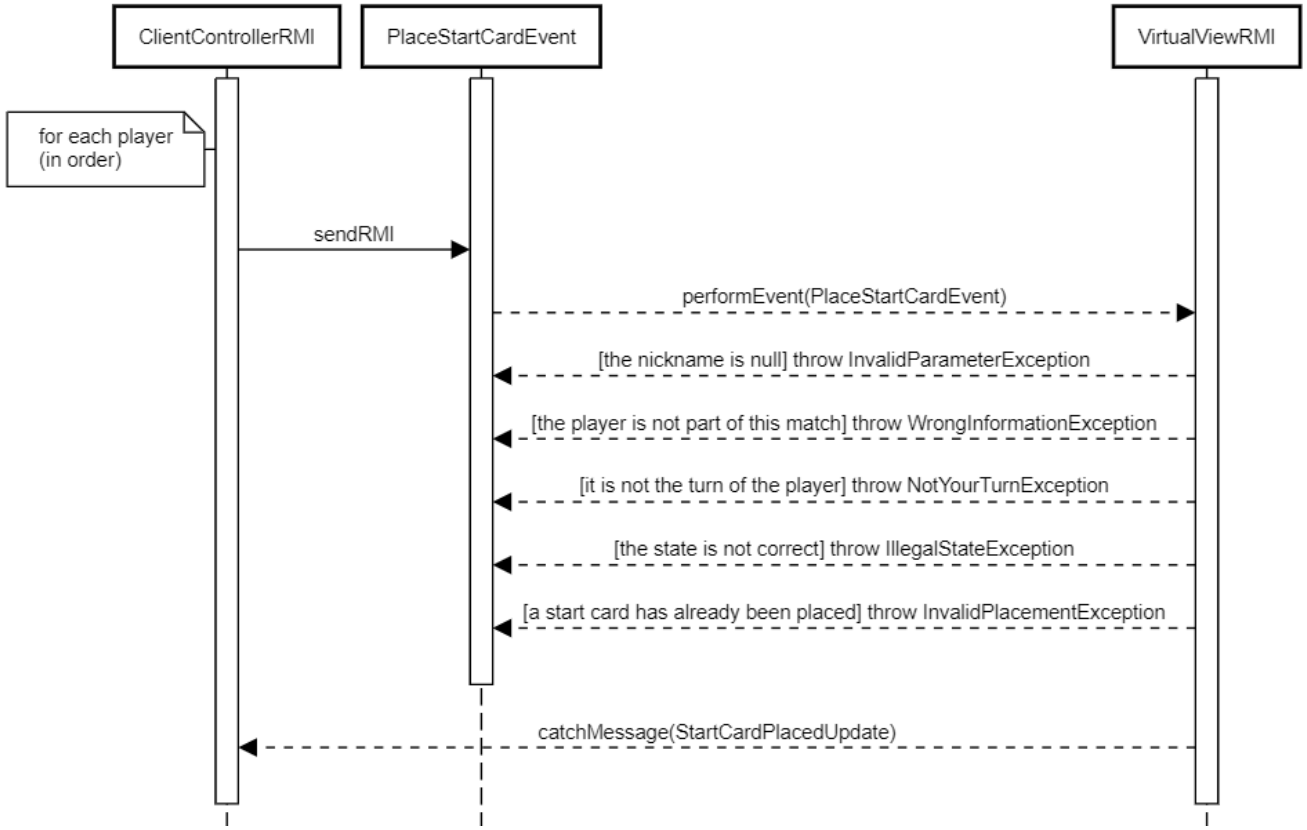
Join a match



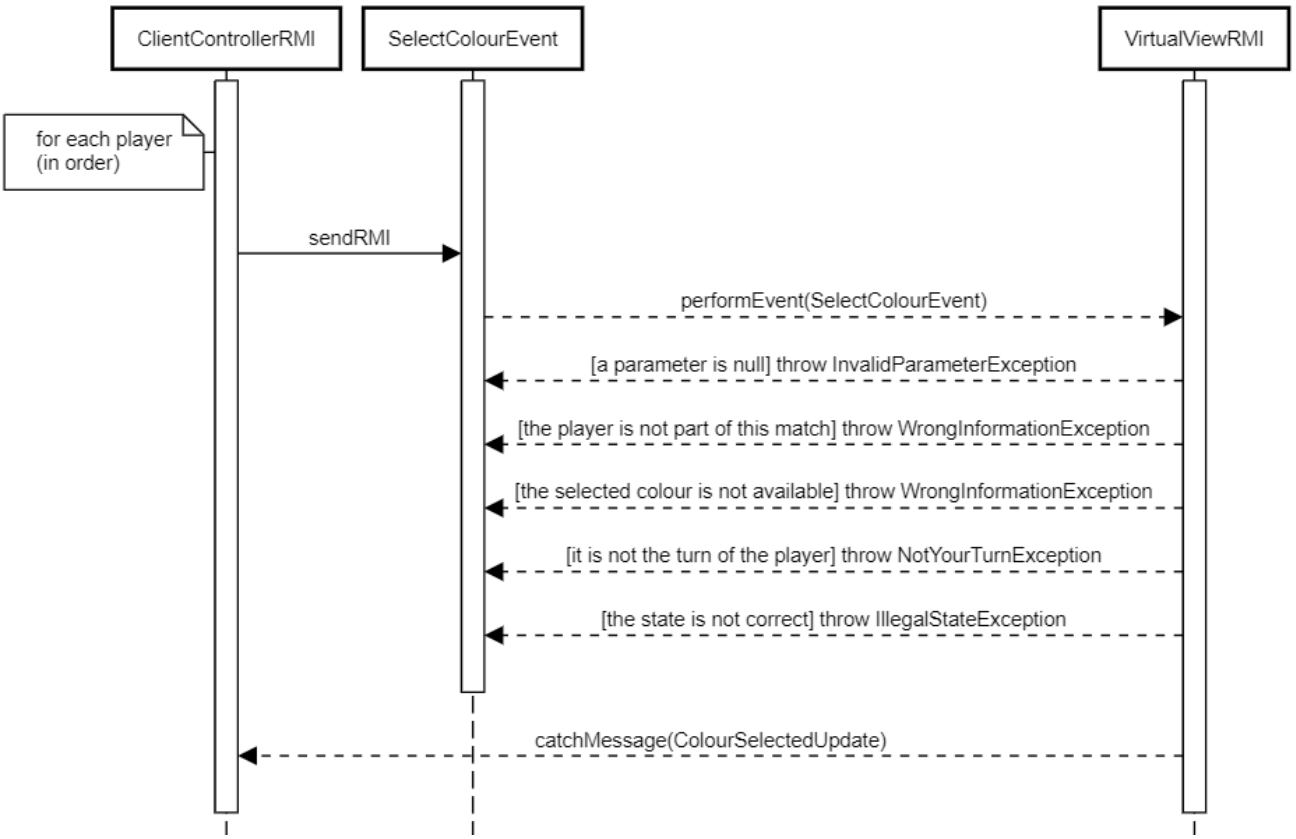
Start match



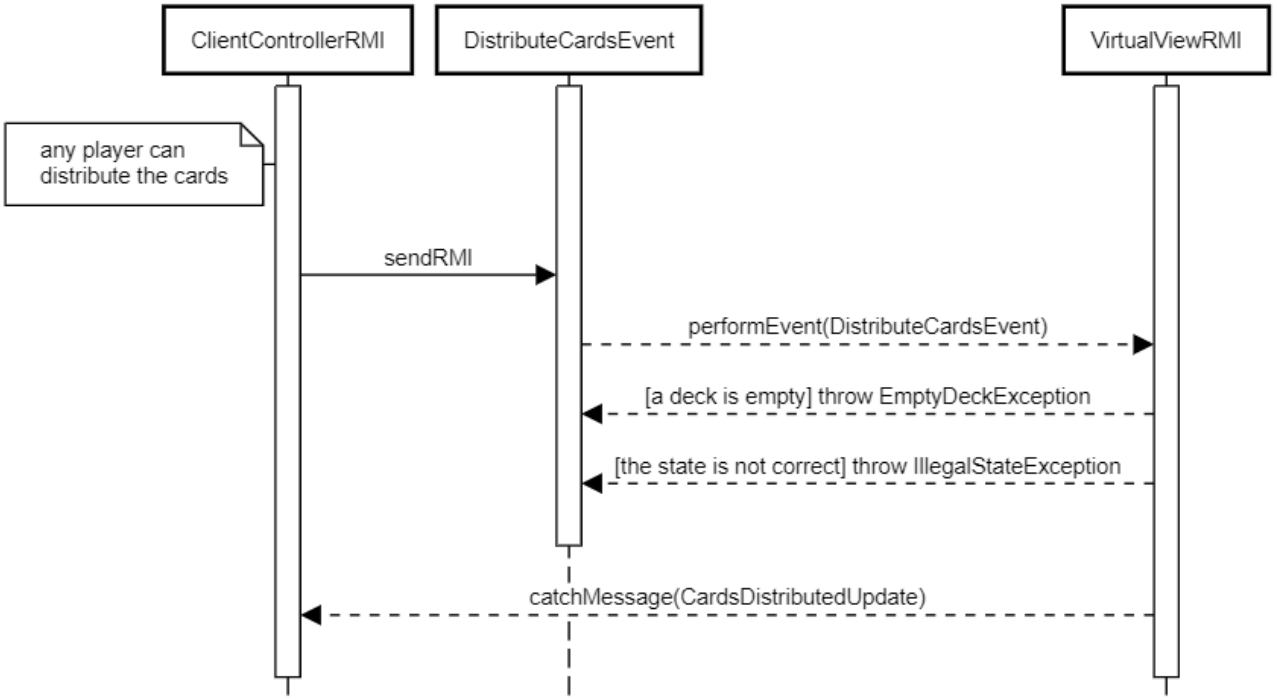
Place start card



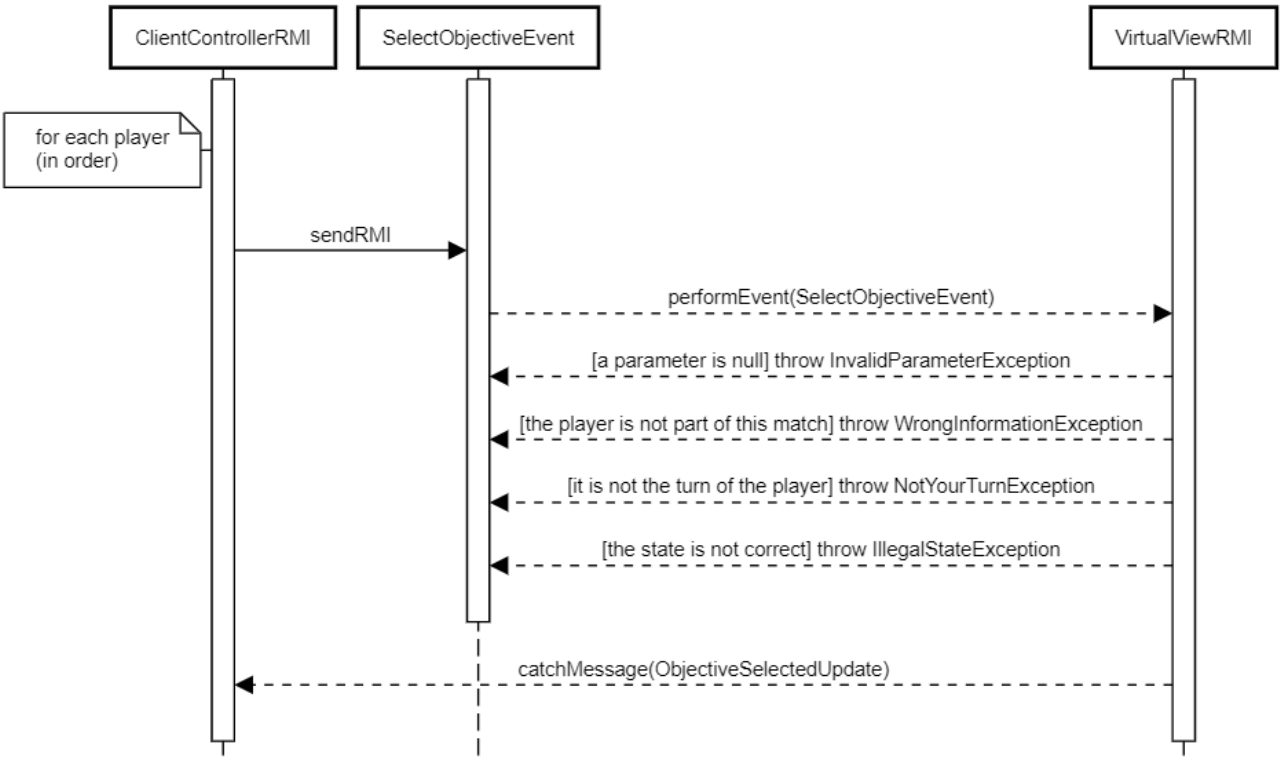
Select colour



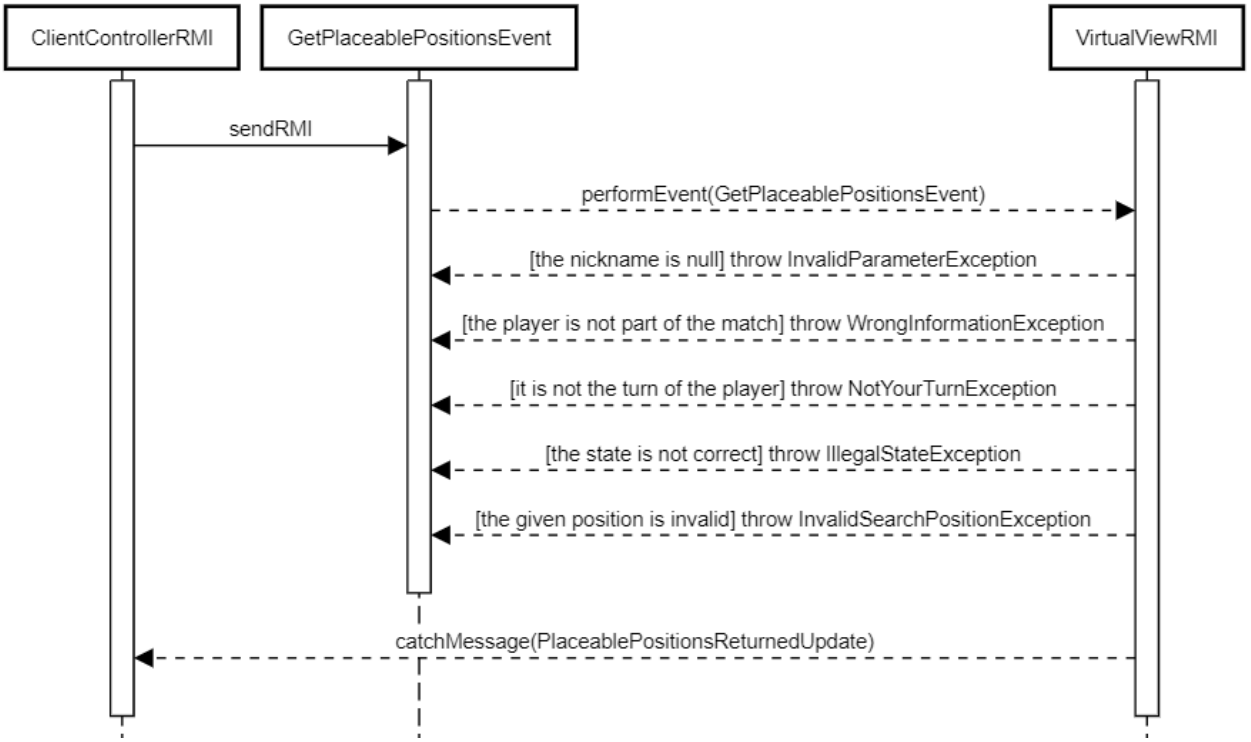
Distribute cards



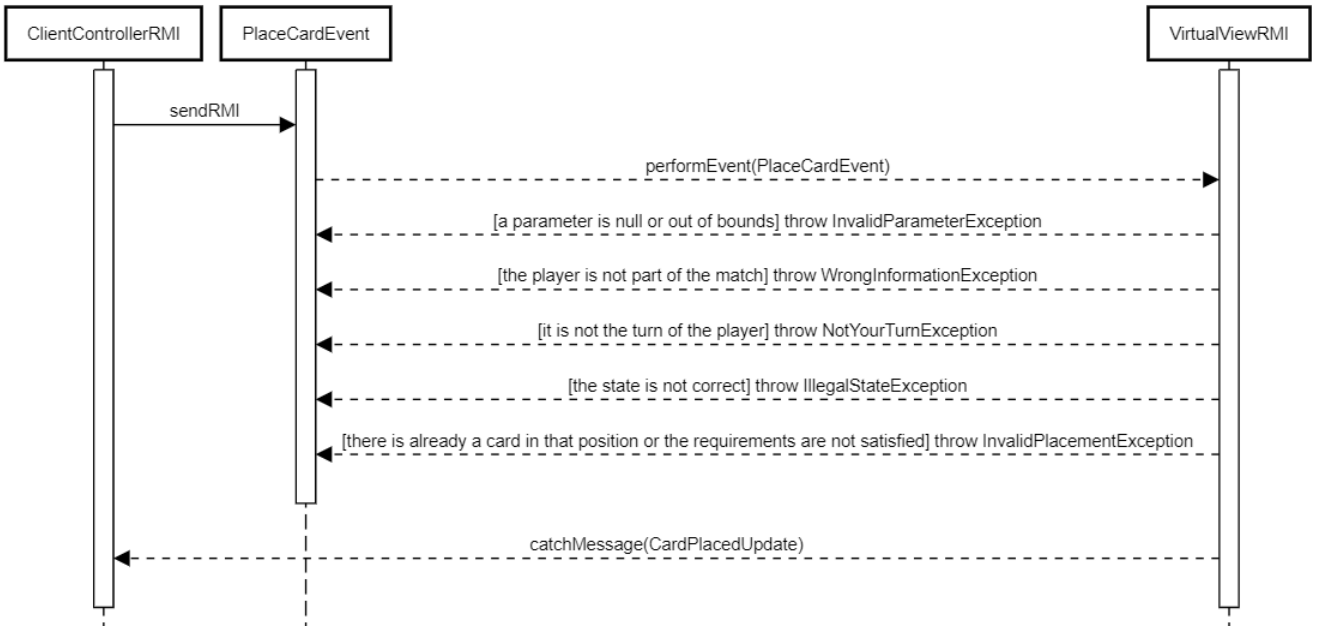
Select secret objective



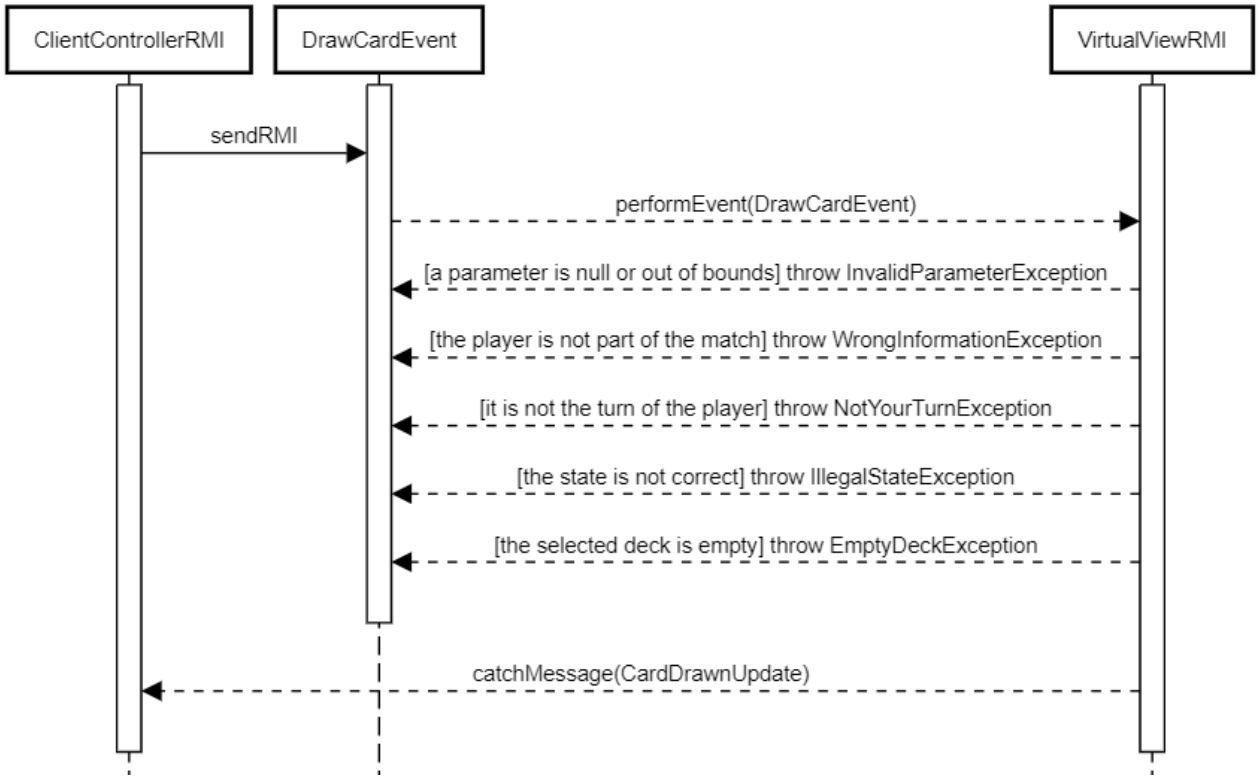
Get Placeable Positions



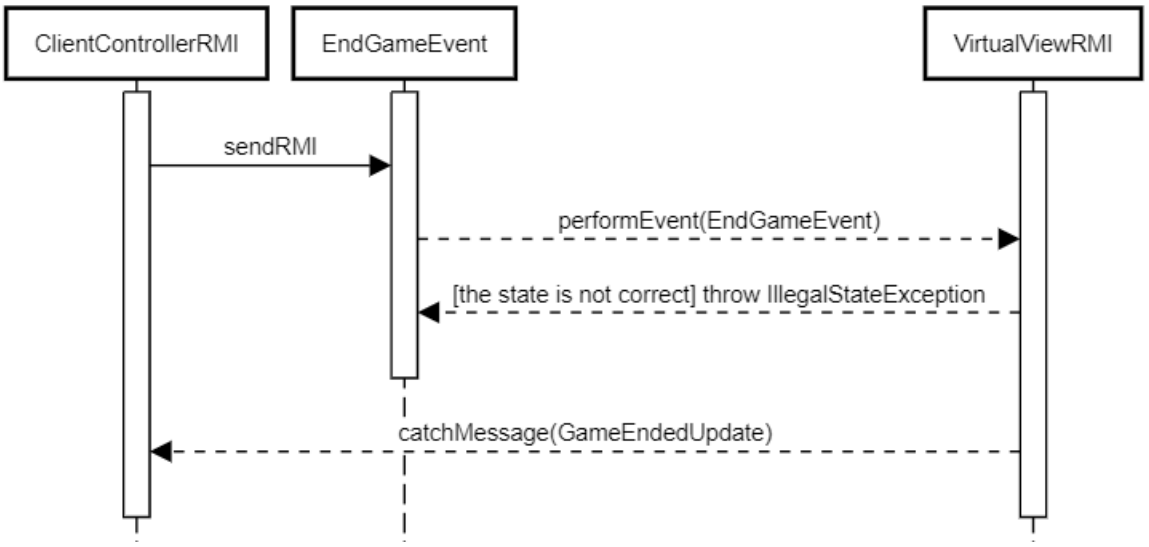
Place a card



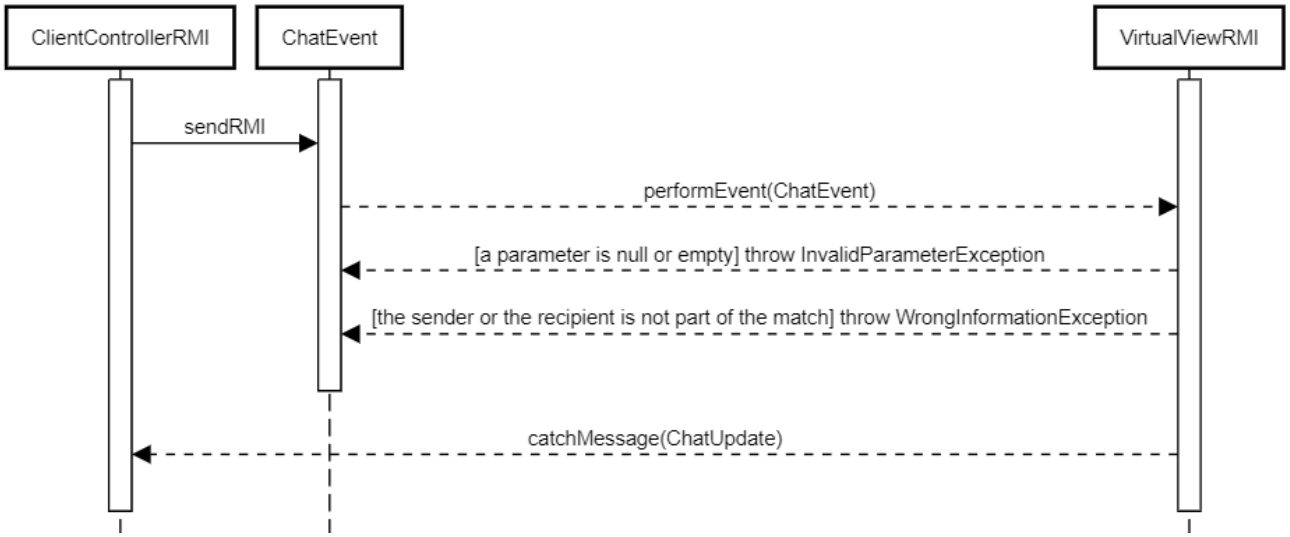
Draw a card



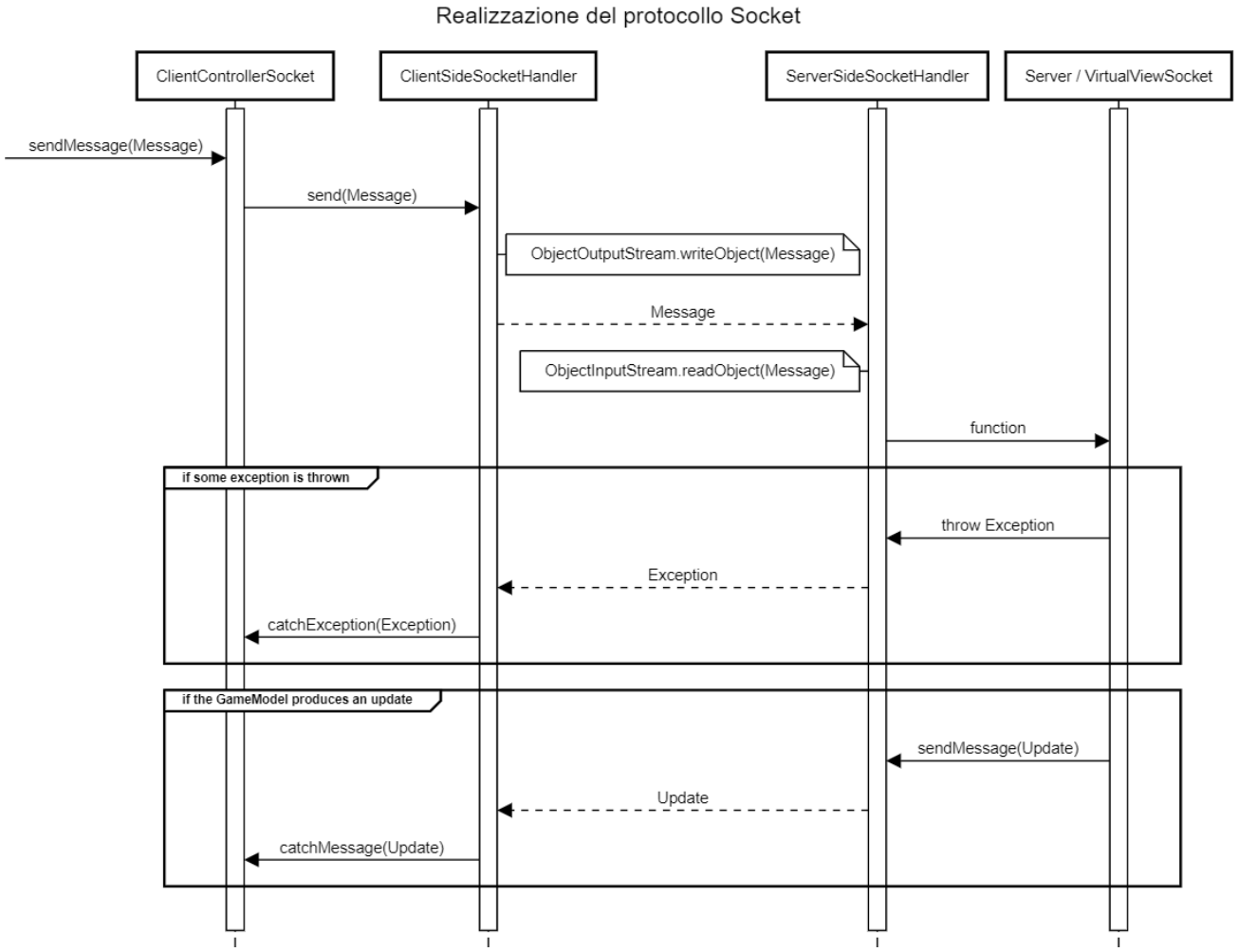
Get classification



Send a message in the chat



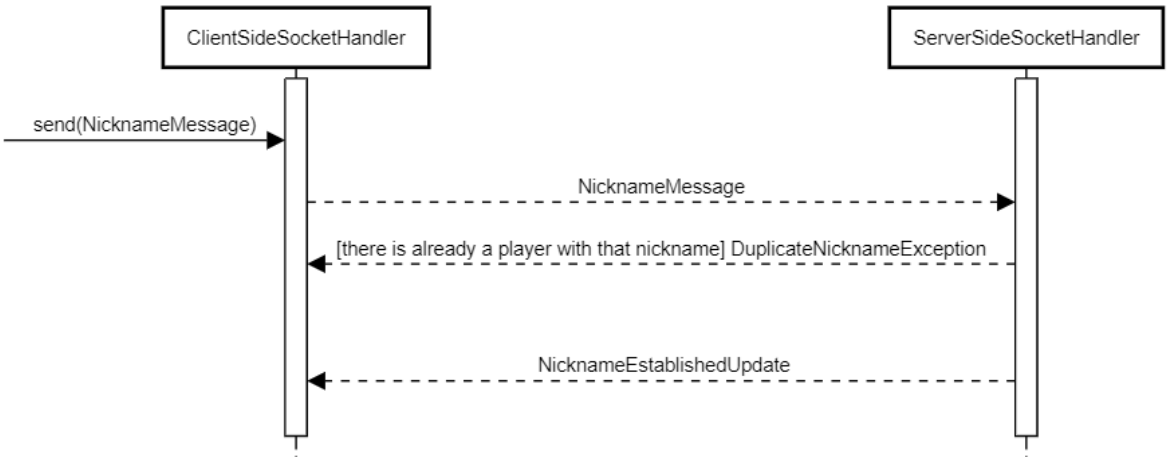
Protocollo Socket



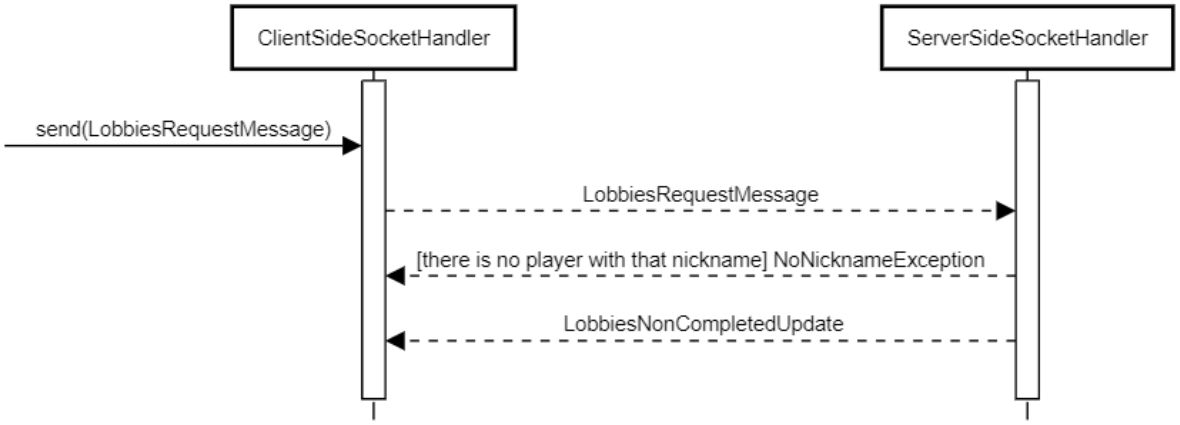
Nel protocollo Socket, i messaggi sono scambiati sulla rete come oggetti. Rispetto al protocollo RMI, sono state inserite due classi (*ClientSideSocketHandler* e *ServerSideSocketHandler*) adibite allo scambio dei messaggi sulla rete.
Le funzioni invocate dal *ServerSideSocketHandler*, indicate nel diagramma con l'etichetta generica "function", dipendono dal tipo di messaggio ricevuto e corrispondono a quelle già illustrate per il protocollo RMI.

I diagrammi seguenti mostrano, per ogni fase del gioco, gli oggetti scambiati attraverso la rete.

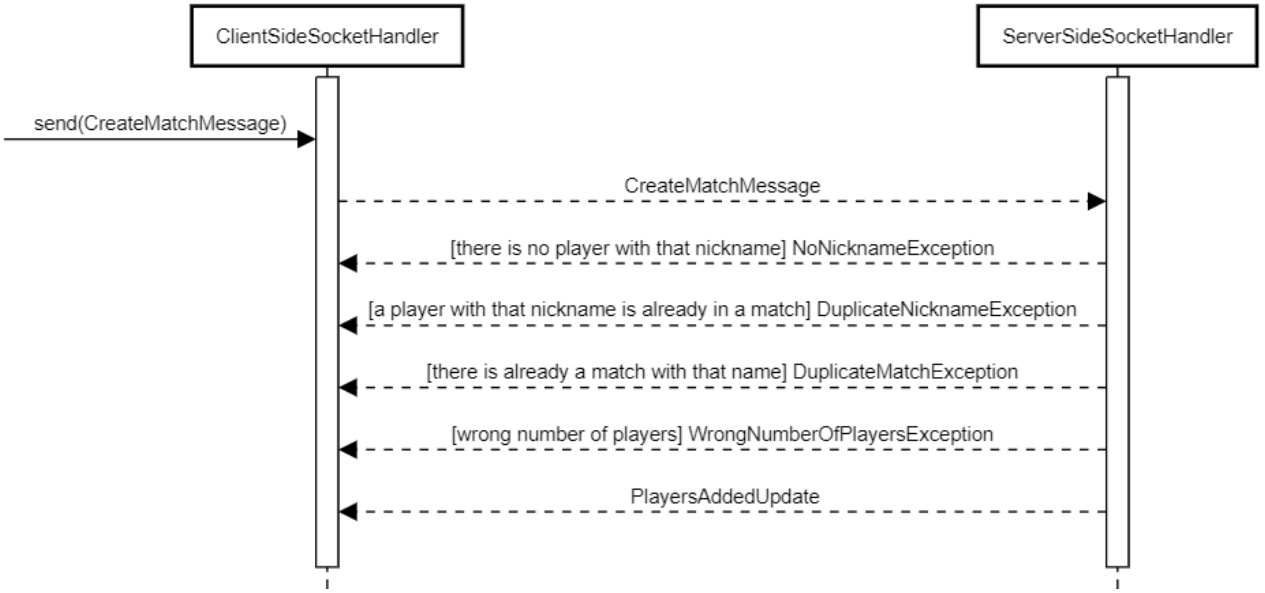
Set nickname



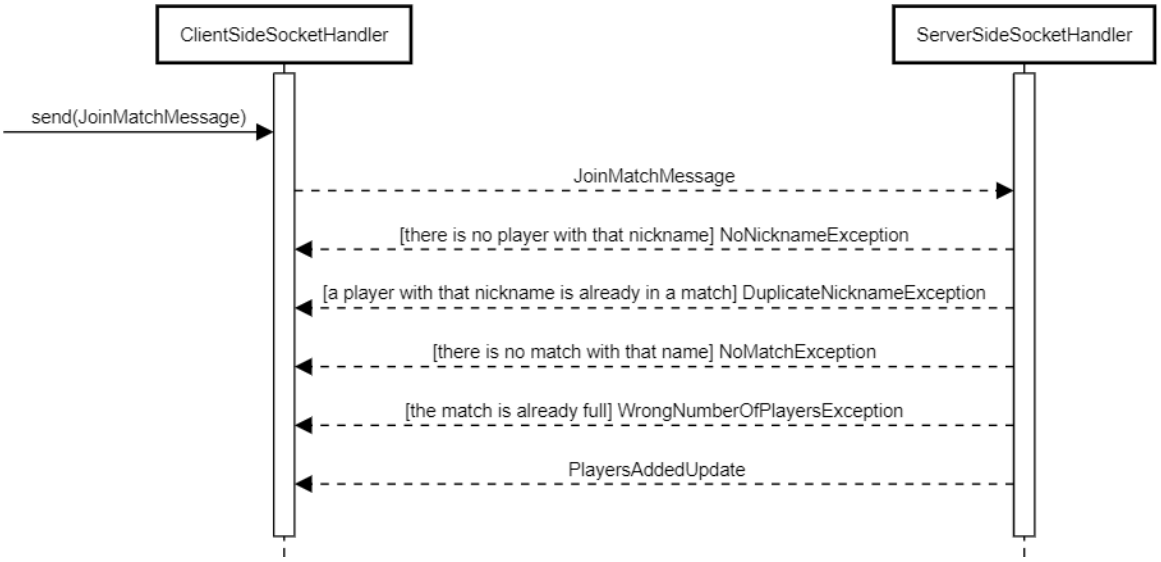
Get Incomplete Lobbies



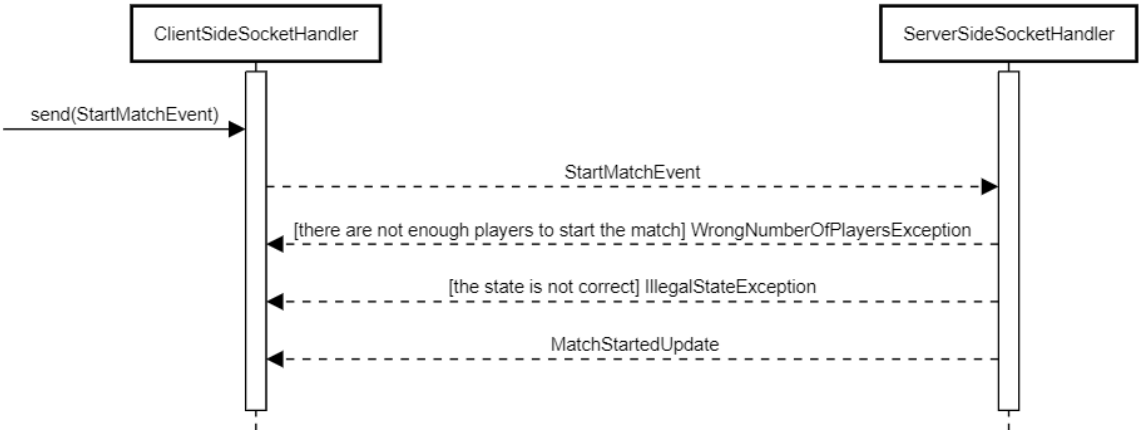
Create a match



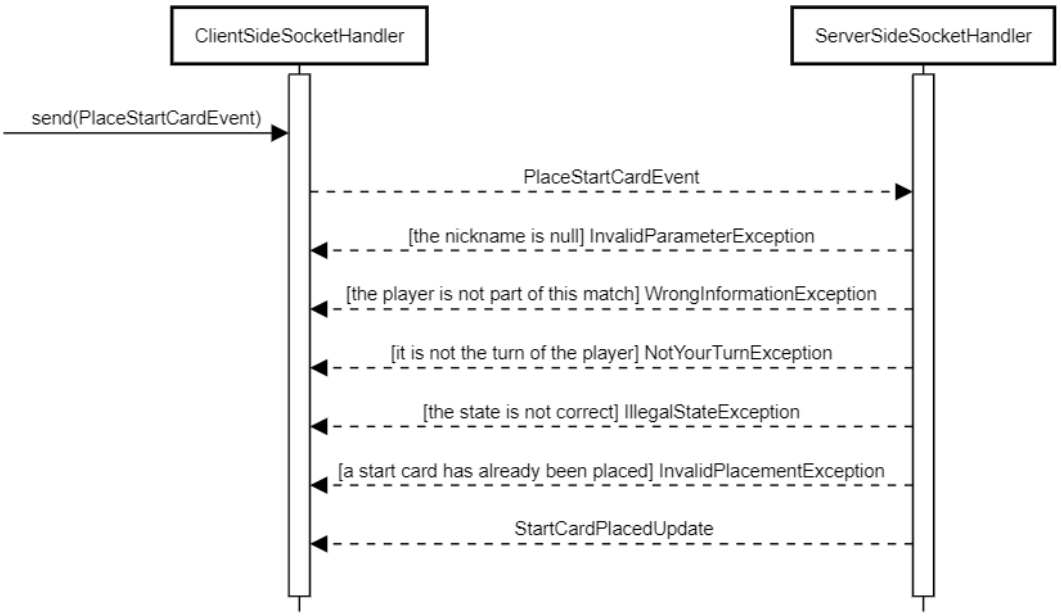
Join a match



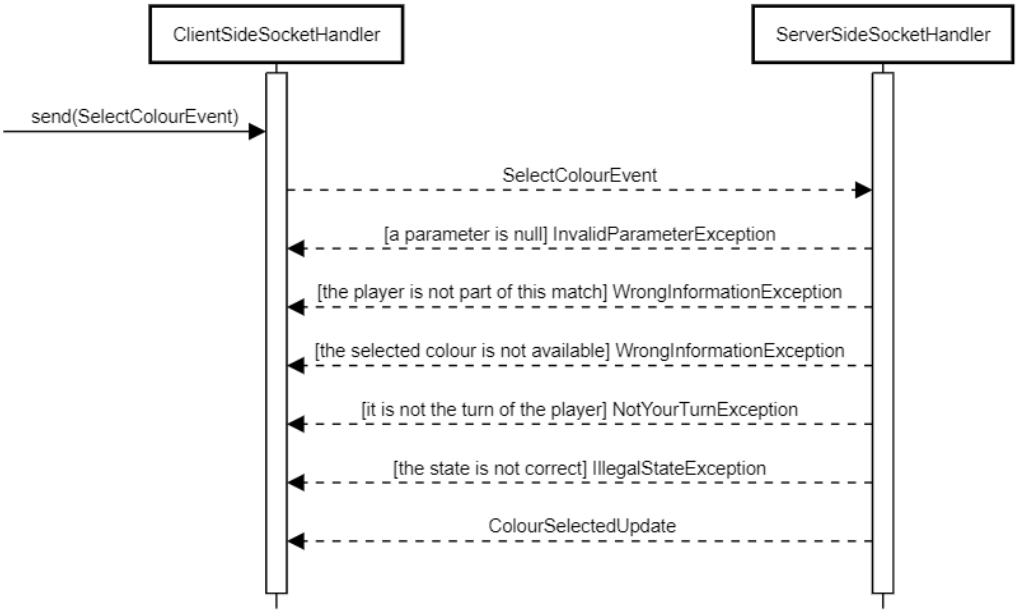
Start match



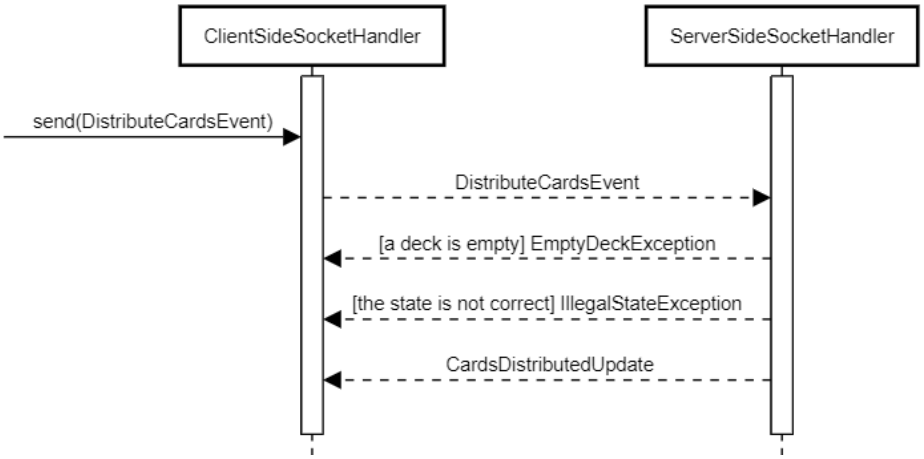
Place start card



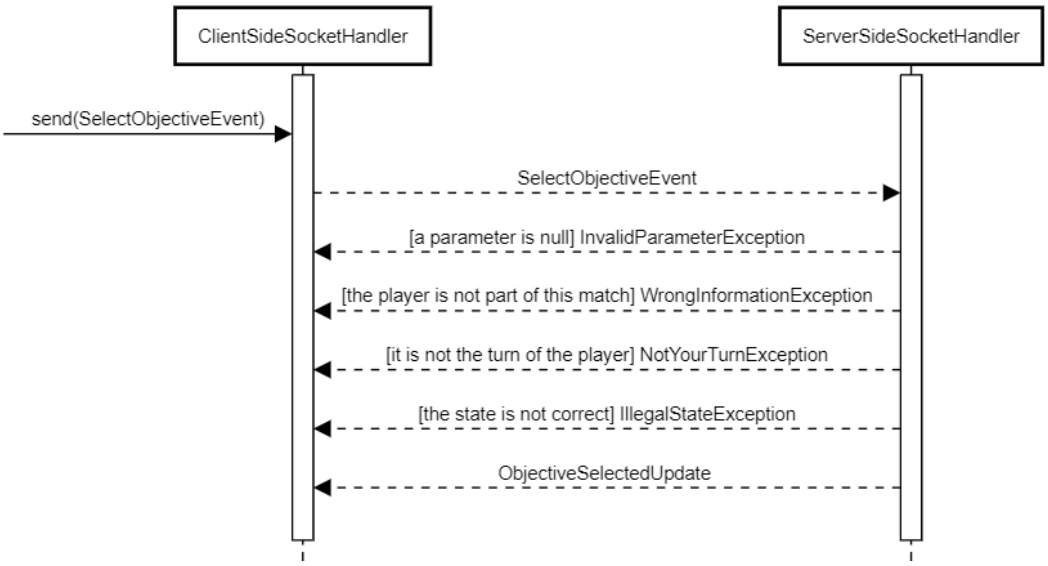
Select colour



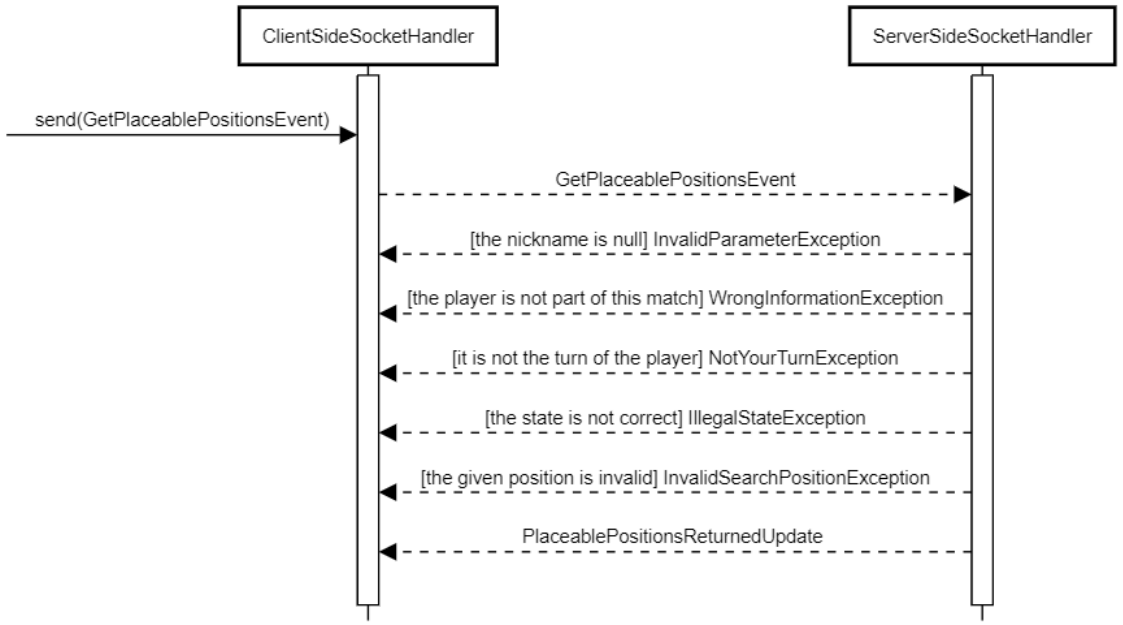
Distribute cards



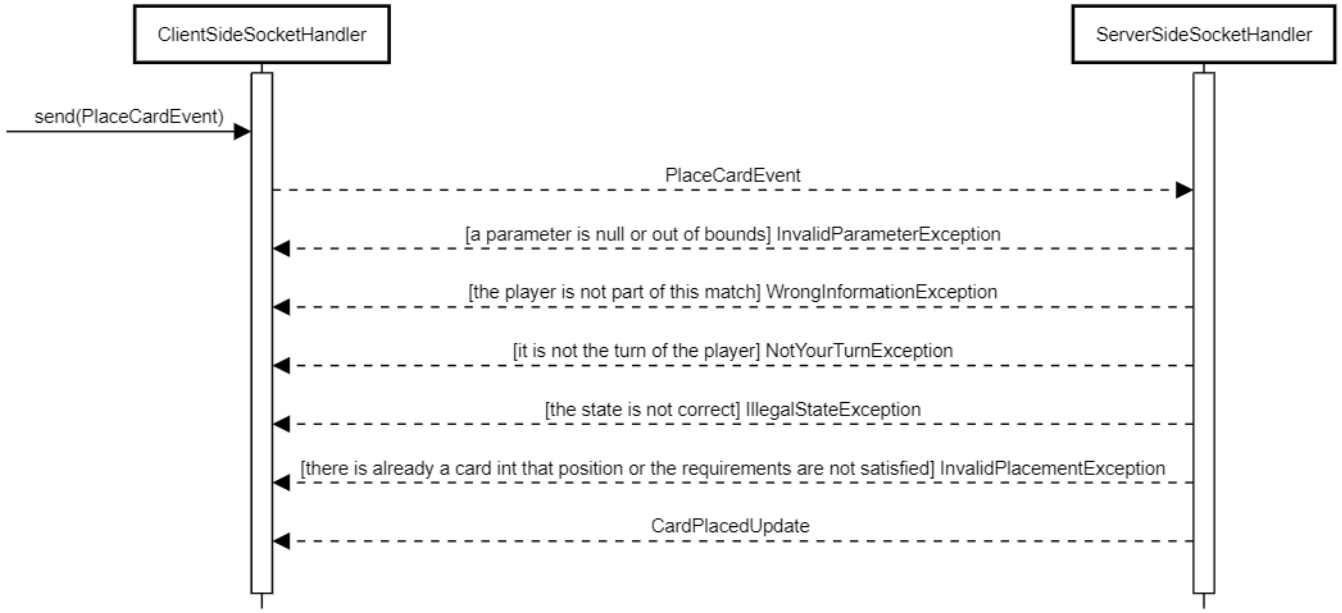
Select secret objective



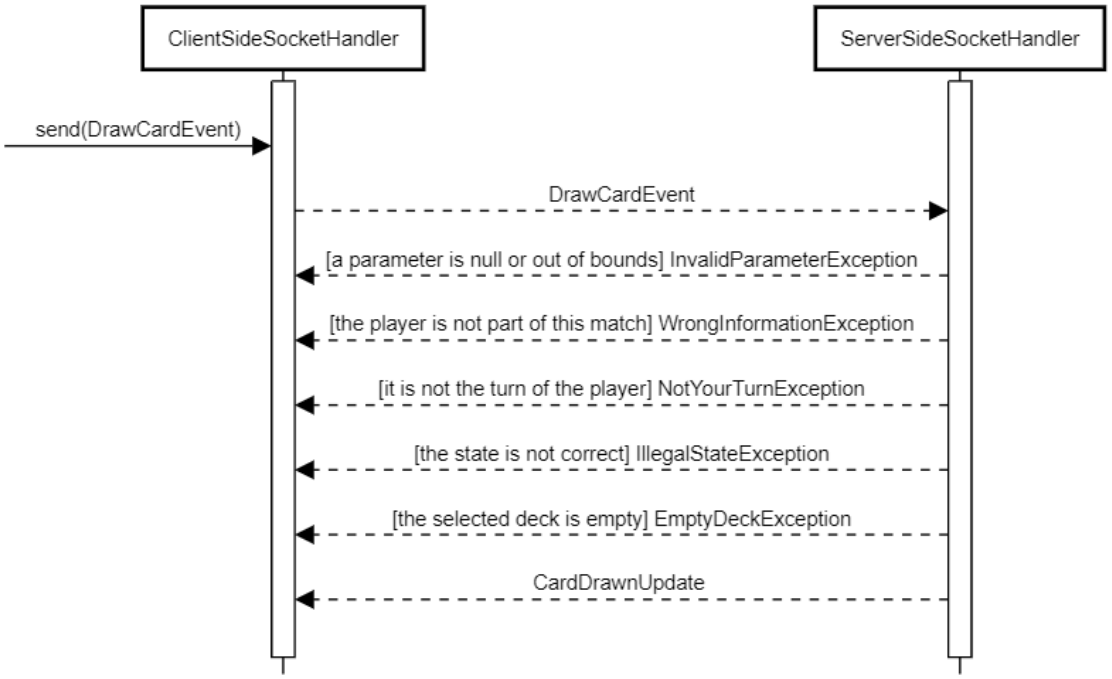
Get Placeable Positions



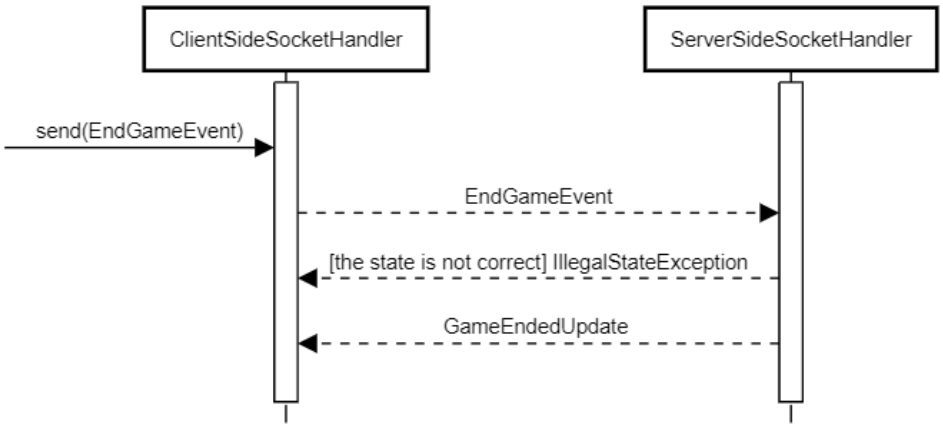
Place a card



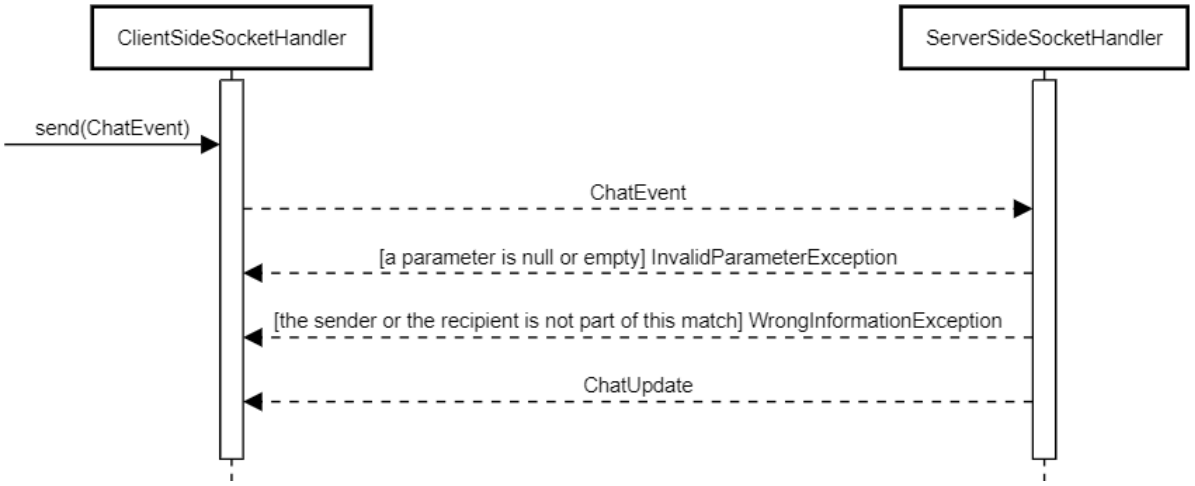
Draw a card



Get classification

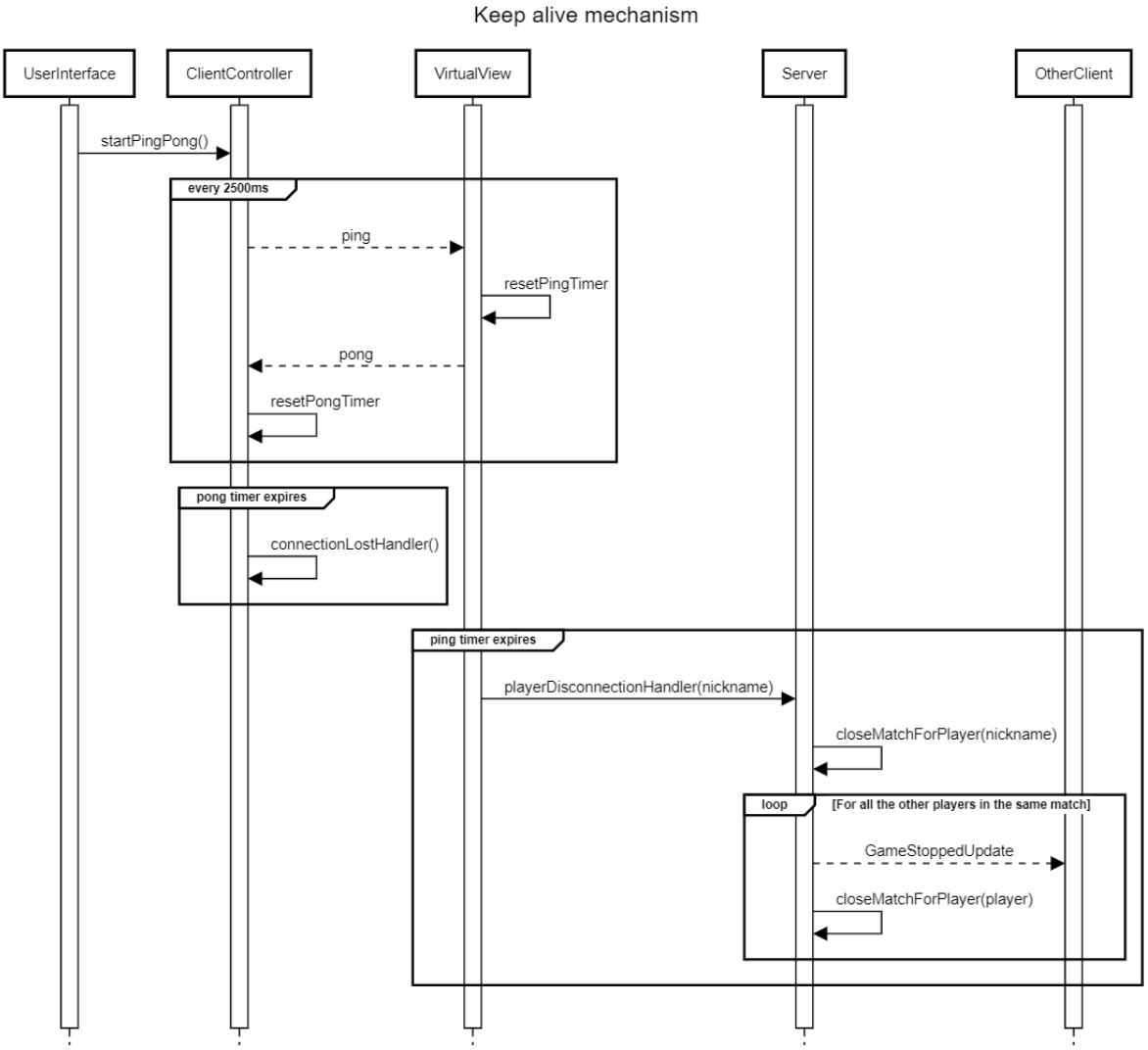


Send a message in the chat



Disconnessione volontaria e involontaria

Per verificare che un client sia connesso, abbiamo previsto un meccanismo di ping e pong tra client e server, che viene attivato in un thread apposito quando un giocatore imposta il suo nickname.
Il ClientController invia ping alla VirtualView ogni 2,5 secondi: se non riceve un pong entro 10s, considera la connessione con il server persa e chiude la partita.
Se la VirtualView non riceve un ping dal client per 10s, considera il client disconnesso e chiude la partita (anche per gli altri giocatori).



I ping e pong sono implementati in modo che la gestione sia il più possibile simile tra i due protocolli di comunicazione presenti.
Protocollo RMI: il ping è la funzione *invokePingRMI()* dell'interfaccia *VVStub*; il pong è la funzione *invokePongRMI()* dell'interfaccia *ClientStub.*
Protocollo Socket: ping e pong sono oggetti di tipo stringa scambiati tra ClientSideSocketHandler e ServerSideSocketHandler.
La gestione del meccanismo e dei timer è comune ad entrambi i protocolli, con il metodo responsabile dell'invio dei ping *pingServer()* in override nelle rispettive sottoclassi di ClientController.

Un client può anche disconnettersi volontariamente, inviando al Server un messaggio di tipo *CloseMatchConnectionMessage*.

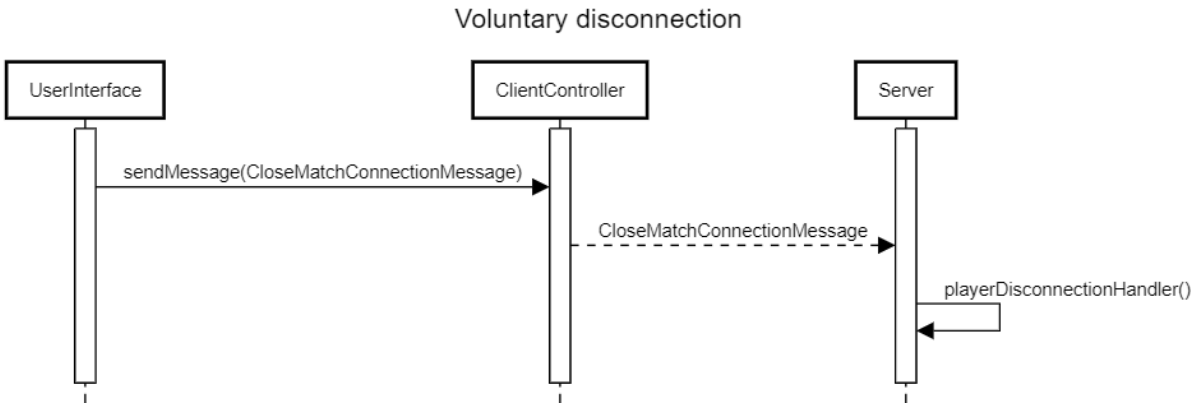


Tabella dei messaggi

Message	**Update**
NicknameMessage: - nickname - clientStub	**NicknameEstablishedUpdate**: - nickname
LobbiesRequestMessage	**LobbiesNonCompletedUpdate**: - non completed lobbies with the number of available spots
CreateMatchMessage: - name of the match - number of players	//
JoinMatchMessage: - name of the match	//
CloseMatchConnectionMessage	**GameStoppedUpdate**

Event	**Update**
JoinMatchEvent: - nickname - virtualView	**PlayersAddedUpdate:** - list of nicknames in the lobby - state (LOBBY or INITIALIZATION)
StartMatchEvent	**MatchStartedUpdate:** - list of players - assigned start cards - colour of the top card of each deck - public cards - player in turn - state (STARTCARD)
PlaceStartCardEvent: - nickname - selected side	**StartCardPlacedUpdate:** - nickname of the player who placed - which card was placed and on which side - player in turn - state (STARTCARD or COLOUR)
SelectColourEvent: - nickname - selected colour	**ColourSelectedUpdate:** - nickname of the player who selected a colour - which colour - player in turn - state (COLOUR or DISTRIBUTION)

DistributeCardsEvent	**CardsDistributedUpdate:** - cards distributed to each player - secret objectives to choose - public objectives - colour of the top card of each deck - player in turn - state (OBJECTIVE)
SelectObjectiveEvent: - nickname - first or second objective	**ObjectiveSelectedUpdate:** - nickname of the player who selected his objective - which objective - player in turn - state (OBJECTIVE or PLACING)
GetPlaceablePositionsEvent: - nickname - coordinates of the card around which the player wants to know where it's possible to place	**PlaceablePositionsReturnedUpdate:** - nickname of the player who requested the available positions - the available positions
PlaceCardEvent: - nickname - index of the card to place - coordinates where to place - selected side	**CardPlacedUpdate:** - nickname of the player who placed - which card, on which side, in which position - points gained - state (DRAWING)
DrawCardEvent: - nickname - index of the deck from which the player wants to draw a card	**CardDrawnUpdate:** - nickname of the player who draws a card - which card - new public card - new colour of the deck - remaining rounds - player in turn - state (PLACING or END)
EndGameEvent	**GameEndedUpdate:** - number of winners - classification - state (CLOSED)
ChatEvent: - sender - recipient - public or private - content of the message	**ChatUpdate:** - sender - public or private - content of the message